

Matemáticas para la IA

Curso introductorio completo

Vectores · Matrices · Transformaciones Lineales · Redes Neuronales · Backpropagation

Material de estudio

Índice

PARTE I: FUNDAMENTOS DE ÁLGEBRA LINEAL

1. Vectores: información en forma de flecha
2. Matrices: tablas que transforman
3. Transformaciones lineales
4. Compatibilidad: filas vs columnas
5. Combinaciones lineales y span
6. Dependencia e independencia lineal
7. Reducción de espacios y pérdida de información
8. Espacio ambiente vs. dimensión intrínseca
9. El determinante y el colapso

PARTE II: DE MATRICES A REDES NEURONALES

10. Matriz $\times$ Vector vs Matriz $\times$ Matriz
11. Una capa de red neuronal, paso a paso
12. Bias y función de activación

PARTE III: CÓMO APRENDE UNA RED

13. La función de pérdida
14. Descenso de gradiente: la intuición
15. Backpropagation: propagar el error
16. Un ciclo completo de entrenamiento
17. Variantes prácticas y problemas típicos

PARTE IV: RESÚMENES Y CHEAT SHEETS

PARTE I: Fundamentos de álgebra lineal

1. Vectores: información en forma de flecha

Un **vector** es una lista ordenada de números. Geométricamente, puede pensarse como una flecha que parte del origen y termina en las coordenadas indicadas.

$$\begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

En Inteligencia Artificial, un vector puede representar **cualquier cosa** descrita con números:

- Una casa: (superficie, habitaciones, baños, antigüedad)
- Una imagen de 28×28 píxeles: vector de 784 dimensiones
- Un embedding de palabra: vector de 768, 1024 o más dimensiones

Idea clave: los vectores son el lenguaje con el que la IA "ve" el mundo.

2. Matrices: tablas que transforman

Una **matriz** es una tabla rectangular de números. Puede leerse de dos formas:

- **Vista almacenamiento:** guarda datos organizados. Ejemplo: 1000 casas × 4 características.
- **Vista acción:** es una **máquina que transforma vectores**. Recibe un vector y devuelve otro.

$$A \cdot v = w$$

Truco clave: las columnas nos dicen a dónde van a parar los vectores base. La primera columna es donde termina (1,0); la segunda, donde termina (0,1).

3. Transformaciones lineales

Una transformación lineal mueve los vectores respetando dos reglas:

- Las líneas rectas siguen siendo rectas (no se curvan).
- El origen no se mueve.

Puede **rotar**, **estirar**, **comprimir** o **reflejar** el espacio, pero no puede curvarlo. **Toda transformación lineal se representa con una matriz.**

4. Compatibilidad: filas vs columnas

Si A es una matriz de tamaño $m \times n$:

$$A_{(m \times n)} \cdot v_{(n \times 1)} = w_{(m \times 1)}$$

Regla: las columnas de la matriz deben coincidir con las componentes del vector.

- Columnas (n) = dimensión de entrada
- Filas (m) = dimensión de salida

¿Qué representa cada fila? Cada fila es una "receta" que combina los números del vector de entrada para producir **una sola componente** de la salida. En una red neuronal, **cada fila es una neurona**.

Ejemplo de cálculo

$$A = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 3 & 4 \end{bmatrix} \quad v = \begin{bmatrix} 5 \\ 1 \\ 2 \end{bmatrix} \quad Av = \begin{bmatrix} 11 \\ 11 \end{bmatrix}$$

$$\text{Fila 1: } 2 \cdot 5 + 1 \cdot 1 + 0 \cdot 2 = 11 \quad | \quad \text{Fila 2: } 0 \cdot 5 + 3 \cdot 1 + 4 \cdot 2 = 11$$

5. Combinaciones lineales y span

Una **combinación lineal** de vectores v_1 y v_2 es:

$$a \cdot v_1 + b \cdot v_2$$

El **span** es el conjunto de **todos** los vectores construibles por combinaciones lineales.

- Span de 1 vector no nulo en 2D: una **recta**.
- Span de 2 vectores en direcciones distintas en 2D: **todo el plano**.
- Span de 3 vectores bien orientados en 3D: **todo el espacio**.

6. Dependencia e independencia lineal

Linealmente independientes: ninguno se puede escribir como combinación de los otros. Cada uno aporta una dirección nueva.

Linealmente dependientes: al menos uno es redundante: se puede construir a partir de los demás.

$$v_1 =$$

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$v_2 =$$

$$\begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

$$\rightarrow v_2 = 2 \cdot v_1$$

(dependientes)

7. Reducción de espacios: pérdida de información

Cuando una matriz tiene columnas dependientes, la transformación colapsa dimensiones. El rango es el número de columnas independientes. Coincide con la dimensión del espacio imagen (a dónde llegan los vectores).

Rangos posibles en una matriz 3×3 aplicada a $\mathbb{R}^3$:

Rango	Cols. indep.	Se transforma en...	Info perdida
3	3	Todo $\mathbb{R}^3$ (sin colapso)	Ninguna
2	2	Un plano	1 dimensión
1	1	Una recta	2 dimensiones
0	0	Solo el origen	Todo

8. Espacio ambiente vs. dimensión intrínseca

Un vector puede tener muchas componentes pero moverse en un subespacio pequeño.

Analogía: una hoja de papel dentro de una habitación 3D. Cada esquina se describe con 3 coordenadas, pero la hoja es bidimensional. Vive en 3D pero tiene solo 2 grados de libertad.

Con una matriz 4×2, entra un vector 2D y sale uno con 4 componentes. Pero la imagen es **un plano 2D dentro de $\mathbb{R}^4$** : solo 2 de las 4 componentes son libres, las otras 2 están forzadas.

Regla general del rango: $\text{rango}(A) \leq \min(m, n)$. No se puede crear información nueva.

9. El determinante y el colapso

El **determinante** mide cómo cambia el volumen del espacio bajo la transformación.

- **det $\neq 0$** : no hay colapso. La transformación se puede deshacer (existe inversa).
- **det = 0**: hay colapso. El volumen se aplasta. La transformación no se puede deshacer.

Importante: det = 0 **no significa** que todo se aplaste al origen. Significa que el espacio se aplasta a un subespacio de menor dimensión. Cuánta dimensión pierde lo dice el rango.

El determinante es el semáforo (¿hay colapso?), el rango es el velocímetro (¿cuánto colapso?).

PARTE II: De matrices a redes neuronales

10. Matriz × Vector vs Matriz × Matriz

Cuando en lugar de un solo vector procesamos varios a la vez, los **apilamos como columnas de una matriz**. La multiplicación matriz × matriz no es una operación nueva: es hacer muchas multiplicaciones matriz × vector al mismo tiempo.

El elemento (i,j) del producto $A \cdot B$ es: fila i de A · columna j de B.

Cada columna del resultado es un vector de entrada transformado. Cada columna de B representa un dato distinto, no una parte del mismo dato.

¿Por qué importa esto en IA? Cuando entrenamos una red, procesamos los datos en **lotes (batches)**. Si apilamos 64 ejemplos como columnas de una matriz X, entonces $W \cdot X$ procesa las 64 muestras al mismo tiempo. Los GPUs están diseñados justamente para hacer estas multiplicaciones enormes en paralelo.

11. Una capa de red neuronal, paso a paso

Ejemplo: red que aprueba préstamos

Cada solicitante se describe con 4 características (normalizadas entre 0 y 1):

$$x = \begin{bmatrix} 0.35 \\ 0.45 \\ 0.10 \\ 0.60 \end{bmatrix} \quad (\text{edad, ingreso, deudas, años trabajo})$$

Diseñamos una capa oculta con **3 neuronas**. La matriz de pesos es 3×4 :

$$W = \begin{bmatrix} 0.2 & 0.8 & -0.5 & 0.3 \\ -0.4 & 0.1 & -0.9 & 0.2 \\ 0.6 & 0.5 & -0.2 & 0.7 \end{bmatrix} \quad b = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.05 \end{bmatrix}$$

Interpretación de cada fila (neurona)

Neurona 1 (0.2, 0.8, -0.5, 0.3): valora mucho el ingreso (peso 0.8), penaliza las deudas (-0.5). Detecta "estabilidad económica".

Neurona 2 (-0.4, 0.1, -0.9, 0.2): peso muy negativo para deudas (-0.9). Es un "alarmista de la deuda": se apaga cuando hay deudas altas.

Neurona 3 (0.6, 0.5, -0.2, 0.7): valora positivamente edad, ingreso y estabilidad laboral. Detecta un "perfil balanceado".

Cálculo neurona por neurona

$$z_{\blacksquare} = 0.2 \cdot 0.35 + 0.8 \cdot 0.45 + (-0.5) \cdot 0.10 + 0.3 \cdot 0.60 + 0.1 = \mathbf{0.66}$$

$$z_{\blacksquare} = (-0.4) \cdot 0.35 + 0.1 \cdot 0.45 + (-0.9) \cdot 0.10 + 0.2 \cdot 0.60 + (-0.2) = \mathbf{-0.265}$$

$$z_{\blacksquare} = 0.6 \cdot 0.35 + 0.5 \cdot 0.45 + (-0.2) \cdot 0.10 + 0.7 \cdot 0.60 + 0.05 = \mathbf{0.885}$$

¿De dónde salieron los pesos?

Nadie los eligió a mano. La red los aprendió sola durante el entrenamiento, comenzando con valores aleatorios y ajustándolos miles o millones de veces hasta que reflejaron patrones estadísticos presentes en los datos históricos. Que las deudas tengan peso negativo no fue programado: emergió porque en los datos, los solicitantes con muchas deudas tendían a no pagar.

12. Bias y función de activación

El **bias (b)** es un número extra que se suma a cada neurona. Permite que la neurona se "encienda" incluso con entradas cero, desplazando el punto de activación.

La **función de activación** es una función *no lineal* que se aplica al resultado de cada neurona. La más popular es **ReLU**:

$$\text{ReLU}(x) = \max(0, x)$$

Aplicada a nuestro $z = (0.66, -0.265, 0.885)$ da $a = (0.66, 0, 0.885)$. La neurona 2 quedó "apagada".

¿Por qué es esencial? Sin no linealidad, apilar muchas capas equivaldría a una sola transformación lineal. La ReLU permite a la red aprender patrones curvos y complejos.

PARTE III: Cómo aprende una red

13. La función de pérdida

Necesitamos medir cuán mal lo hizo la red. Si la red predice $\hat{y}$ y la respuesta correcta es y , una medida simple es el **error cuadrático**:

$$L = (\hat{y} - y)^2$$

Ejemplo: la red predice $\hat{y} = 0.75$ pero la respuesta correcta es $y = 1$. Entonces:

$$L = (0.75 - 1)^2 = 0.0625$$

El objetivo del entrenamiento es minimizar L en promedio sobre todos los ejemplos.

14. Descenso de gradiente: la intuición

Analogía del excursionista: imagínate que estás perdido en una montaña con niebla. No ves lejos, pero podés sentir con los pies en qué dirección el suelo baja más rápido. Tu estrategia es dar un paso en esa dirección, sentir de nuevo, y repetir.

- **La montaña** = función de pérdida L .
- **Tu posición** = valor actual de todos los pesos.
- **El valle** = pesos que minimizan L .
- **Los pies** = las derivadas parciales (el gradiente).

Para cada peso w_i calculamos:

$$\partial L / \partial w_i$$

Este número responde a: **si aumento un poquito este peso, ¿cuánto cambia la pérdida?**

- Derivada **positiva grande**: aumentar el peso empeora las cosas. Hay que bajarlo.
- Derivada **negativa grande**: aumentar el peso mejora las cosas. Hay que subirlo.
- Derivada **casi cero**: el peso no afecta mucho la pérdida en este punto.

La regla de actualización

$$w_{\text{nuevo}} = w_{\text{viejo}} - \eta \cdot (\partial L / \partial w)$$

- **El signo menos** es clave: vamos en contra del gradiente para bajar la pérdida.
- η (**eta**) es la **tasa de aprendizaje**: el tamaño del paso (típicamente 0.01 o 0.001).

Ejemplo micro: $w = 0.5$, $\partial L / \partial w = -0.25$, $\eta = 0.1$

$$w_{\text{nuevo}} = 0.5 - 0.1 \cdot (-0.25) = 0.5 + 0.025 = 0.525$$

El peso subió un poquito. Repetimos con el nuevo peso, calculamos nueva derivada, actualizamos otra vez. Miles o millones de veces.

15. Backpropagation: propagar el error hacia atrás

El problema: en una red profunda hay millones de pesos, y cada uno afecta la pérdida indirectamente pasando por muchas capas. ¿Cómo calculamos $\partial L / \partial w$ para un peso enterrado en la capa 3, si entre él y la pérdida hay 20 capas más?

La solución: backpropagation, un algoritmo que usa la **regla de la cadena** para calcular todos los gradientes de forma eficiente.

La regla de la cadena

Si L depende de a , y a depende de w :

$$\partial L / \partial w = (\partial L / \partial a) \cdot (\partial a / \partial w)$$

Intuición: si el precio de la nafta afecta al transporte, y el transporte afecta al precio de la pizza, entonces la nafta afecta a la pizza **multiplicando** los dos efectos.

El algoritmo en dos fases

Fase 1 - Forward pass: se pasa el vector de entrada por la red hacia adelante, guardando todos los valores intermedios. Al final se obtiene la predicción y la pérdida.

Fase 2 - Backward pass: se recorre la red hacia atrás calculando los gradientes:

1. Se calcula $\partial L / \partial \mathbf{z}$ (fácil, es la derivada de la pérdida final).
2. Con la regla de la cadena, se obtiene el gradiente respecto a la última capa.
3. Se calculan los gradientes de esa matriz de pesos (para actualizarla).
4. Se propaga el error hacia atrás a la capa anterior.
5. Se repite capa por capa hasta la primera.

La magia: en cada capa se reutilizan los cálculos de la capa siguiente. Todo el trabajo se hace en un único barrido hacia atrás. Esto hace a backpropagation extraordinariamente eficiente.

Analogía: el equipo culpando hacia atrás

Un CEO recibe una crítica y le pregunta a sus VPs quién contribuyó. Cada VP pregunta a sus gerentes, cada gerente a sus supervisores, hasta los empleados. Cada nivel evalúa cuánto contribuyó cada persona al error, ponderando por su responsabilidad. Al final, cada empleado sabe cuánto de la culpa le toca. Eso es backpropagation: cada peso recibe una "culpa" proporcional a su contribución al error final.

16. Un ciclo completo de entrenamiento

Paso	Acción
1	Tomar un ejemplo (o lote) del dataset.
2	Forward pass: calcular la predicción de la red.
3	Calcular la pérdida comparando con la respuesta correcta.
4	Backward pass: calcular gradientes con backpropagation.
5	Actualizar los pesos con la regla del descenso de gradiente.
6	Repetir con el siguiente ejemplo (miles/millones de veces).

Después de suficientes ciclos, los pesos convergen a valores que hacen que la red rara vez se equivoque en los datos de entrenamiento. Con buenas prácticas, también generaliza bien a datos nuevos.

17. Variantes prácticas y problemas típicos

Variantes del descenso de gradiente

- **SGD (Stochastic Gradient Descent):** usa un solo ejemplo por actualización. Rápido pero ruidoso.
- **Mini-batch:** usa lotes pequeños (32, 64, 128). Balance entre velocidad y estabilidad. Es lo estándar.
- **Momentum:** incorpora "inercia" para acelerar la convergencia.
- **Adam, RMSprop:** ajustan la tasa de aprendizaje adaptativamente para cada peso. Adam es el más popular hoy.

Problemas comunes

- **Gradientes que se desvanecen (vanishing):** en redes muy profundas, los gradientes se vuelven muy pequeños al propagarse, y las primeras capas dejan de aprender.
- **Gradientes que explotan (exploding):** lo opuesto; crecen sin control.
- **Mínimos locales y puntos silla:** el descenso puede atascarse en valles falsos.
- **Overfitting:** la red memoriza los datos de entrenamiento pero falla con datos nuevos.

Cada uno tiene soluciones: mejores funciones de activación, inicialización cuidadosa, batch normalization, dropout, regularización, arquitecturas con conexiones residuales, etc.

PARTE IV: Resúmenes y cheat sheets

Las tres preguntas para leer cualquier matriz

1. ¿Qué dimensión de entrada acepta? → mirá las columnas.
2. ¿Qué dimensión de salida produce? → mirá las filas.
3. ¿Cuánta información conserva realmente? → mirá el rango.

Las tres formas de matriz

Forma	Qué hace	Rango máx.	Uso típico en IA
Cuadrada ($n \times n$)	Transforma sin cambiar dimensión	n	Rotaciones, atención
Alta ($m > n$)	Sumerge en mayor dimensión	n	Embeddings, expansión
Ancha ($m < n$)	Comprime a menor dimensión	m	Encoders, PCA, clasificación

Anatomía de una capa neuronal

$$\mathbf{a} = f(\mathbf{W} \cdot \mathbf{x} + \mathbf{b})$$

- $\mathbf{x}$: vector de entrada (características del dato).
- $\mathbf{W}$: matriz de pesos. Cada fila es una neurona.
- $\mathbf{b}$: vector de bias. Uno por neurona.
- f : función de activación no lineal (ReLU, sigmoide, tanh).
- $\mathbf{a}$: vector de salida. Nueva representación del dato.

Fórmulas clave del entrenamiento

Pérdida: $L = \frac{1}{2} (\hat{y} - y)^2$ (error cuadrático simple)

Gradiente: $\nabla L = (\partial L / \partial \mathbf{w}_1, \partial L / \partial \mathbf{w}_2, \dots, \partial L / \partial \mathbf{w}_n)$

Actualización: $w_{\text{nuevo}} = w_{\text{viejo}} - \eta \cdot (\partial L / \partial w)$

Regla de la cadena: $\partial L / \partial \mathbf{w} = (\partial L / \partial \mathbf{a}) \cdot (\partial \mathbf{a} / \partial \mathbf{w})$

Por qué todo esto importa en IA

- **PCA:** reduce dimensiones eligiendo direcciones que preserven información.
- **Redes neuronales:** apilan capas de transformaciones lineales + no linealidades.
- **Autoencoders:** comprimen datos por un cuello de botella.
- **Embeddings:** representan objetos discretos (palabras) como vectores densos.
- **Transformers:** usan matrices de atención para relacionar tokens en secuencias.
- **Entrenamiento:** ajuste iterativo de pesos vía gradiente + backpropagation.

La gran síntesis

Podemos ver toda la IA moderna como una danza entre dos ideas:

- 1. Representación:** transformar los datos con matrices para que las clases o patrones se vuelvan más fáciles de separar. Cada capa es un "doblado" del espacio.
- 2. Aprendizaje:** ajustar los pesos de esas matrices minimizando una función de pérdida, mediante descenso de gradiente y backpropagation.

Los vectores son el lenguaje. Las matrices son las máquinas que los transforman. Las transformaciones lineales, encadenadas y decoradas con no linealidades, construyen modelos capaces de reconocer imágenes, traducir idiomas y generar texto. Y todo eso se aprende solo, ajustando pesos mediante los pequeños empujones del gradiente.

Próximos temas sugeridos: autovalores y autovectores, descomposición SVD, regularización, redes convolucionales y transformers.